CATALOGUE MANAGEMENT SYSTEM

Full Architectural & Capability Document

Azure + ASP.NET Core + Vue 3 + SQL + EDI/File Integration + DevOps

2. Initial Capability Model

Capability	Why needed	Current coverage	Gap	Criticality
Catalog domain	Core Cybage focus	Unknown	To assess	Critical
Supplier integrations	Core focus; EDI/file exchanges	Unknown	To assess	Critical
File processing / data transformation	EDI/CSV and related exchanges	Unknown	To assess	Critical
Application engineering	Maintenance + new features	Unknown	To assess	Critical
Azure / cloud	Technology transition and operations	Unknown	To assess	Critical
Data / database	Application and data support	Unknown	To assess	High
API / integration	Related applications and supplier connectivity	Unknown	To assess	High
QA / test automation	Quality expectation	Unknown	To assess	High
DevOps / CI-CD	Migration and operational ownership	Unknown	To assess	High
Business analysis / product	Requirements and client alignment	Unknown	To assess	High
HVAC / B2B e-commerce domain	Business understanding	Unknown	To assess	High
Security / identity	Technology separation and operations	Unknown	To assess	Medium
Production support	Steady-state responsiveness	Unknown	To assess	High

Source capability model provided for the project

Purpose

Turn the capability model into a concrete, buildable reference architecture for a realistic catalogue-management learning project. The design intentionally uses free/demo third-party APIs where appropriate and simulates enterprise EDI flows locally so the project can be built without paid supplier access.

Document status	Target stack	Primary deployment
Reference architecture / learning project	Vue 3 + TypeScript, ASP.NET Core 10, EF Core, Azure SQL, Azure Storage, Service Bus, Functions	Azure PaaS / container-ready; local Docker development

Version note

As of 16 September 2026, .NET 10 is the active LTS line; .NET 8 remains in maintenance until 10 November 2026. This document uses .NET 10 for a fresh build while noting .NET 8 compatibility where useful.

Contents

1. Executive summary
2. Capability model and project interpretation
3. Project scope and business capabilities
4. Functional requirements and user journeys
5. Architecture principles and decisions
6. System context architecture
7. Logical application architecture
8. Azure deployment architecture
9. Catalogue domain model and database
10. Supplier integration and EDI architecture
11. File ingestion and transformation pipeline
12. API architecture and contracts
13. Vue frontend architecture
14. Security and identity
15. Testing and mock strategy
16. CI/CD, Docker and environment promotion
17. Observability, support and operations
18. Reliability, resilience and failure handling
19. Performance and scalability
20. Data governance, audit and compliance considerations
21. Free / public third-party APIs and how to use them
22. Sample code and project structure
23. End-to-end scenarios
24. Capability-to-implementation matrix
25. Implementation roadmap
26. Backlog / user stories
27. Architecture risks and trade-offs
28. Definition of done
29. Recommended learning sequence
30. Reference links

How to use this document

Read Sections 6-11 first to understand the architecture end to end. Then use Sections 12-22 while building. Sections 24-29 are the project-management and interview-preparation layer: they map the original capability model to concrete deliverables and show what evidence would demonstrate each capability.

1. Executive summary

The proposed system is a supplier-aware catalogue management platform. It provides a Vue-based administration and catalogue experience, an ASP.NET Core API, an Azure SQL data layer, an import/integration pipeline for supplier files and APIs, asynchronous processing through Azure Service Bus, and operational controls for logging, monitoring, security, audit, and deployment.

The architecture is intentionally modular rather than microservices-first. The core catalogue application is a modular monolith in ASP.NET Core, while file ingestion and long-running supplier processing are separated into event-driven workers/functions. This gives a smaller learning project clear boundaries without creating unnecessary distributed-system overhead.

Target high-level flow



Primary architectural characteristics

- API-first: Vue is a client of the same .NET API used by automated integrations where practical.
- Canonical data model: supplier-specific formats are converted into stable internal DTOs before business rules are applied.
- Asynchronous imports: large files and unreliable external systems are processed through queues rather than holding an HTTP request open.
- Idempotent processing: file hashes, supplier reference keys and import IDs prevent duplicate processing.
- Secure by default: Entra ID or a local development identity abstraction, role-based authorization, Key Vault for secrets, and least-privilege access.
- Observable: correlation IDs, structured logs, traces, metrics, import dashboards, error containers and dead-letter queues.
- Container-ready: every .NET worker/API can run locally through Docker Compose and can be deployed as containers when desired.

2. Capability model and project interpretation

2. Initial Capability Model

Capability	Why needed	Current coverage	Gap	Criticality
Catalog domain	Core Cybage focus	Unknown	To assess	Critical
Supplier integrations	Core focus; EDI/file exchanges	Unknown	To assess	Critical
File processing / data transformation	EDI/CSV and related exchanges	Unknown	To assess	Critical
Application engineering	Maintenance + new features	Unknown	To assess	Critical
Azure / cloud	Technology transition and operations	Unknown	To assess	Critical
Data / database	Application and data support	Unknown	To assess	High
API / integration	Related applications and supplier connectivity	Unknown	To assess	High
QA / test automation	Quality expectation	Unknown	To assess	High
DevOps / CI-CD	Migration and operational ownership	Unknown	To assess	High
Business analysis / product	Requirements and client alignment	Unknown	To assess	High
HVAC / B2B e-commerce domain	Business understanding	Unknown	To assess	High
Security / identity	Technology separation and operations	Unknown	To assess	Medium
Production support	Steady-state responsiveness	Unknown	To assess	High

Original capability model from the supplied image

The source table is an initial capability model, not a final architecture. “Current coverage = Unknown” and “Gap = To assess” indicate that the next task is to convert capability categories into measurable project evidence. The table below translates every capability into concrete implementation work for this project.

Capability	Criticality	What it means in this project	Evidence / deliverables
Catalogue domain	Critical	Product lifecycle, categories, suppliers, attributes, pricing, availability, approvals	Domain model; workflows; product UI; validation rules
Supplier integrations	Critical	External supplier adapters and connectivity	EDI/CSV/REST adapters; supplier registry; mapping configs
File processing / data transformation	Critical	Reliable import of structured files and conversion to canonical data	Blob ingestion; parsing; mapping; validation; reject/quarantine; retry
Application engineering	Critical	Maintainable backend and frontend	ASP.NET Core, Vue, DI, EF Core, API design, clean architecture
Azure / cloud	Critical	Cloud hosting, transition and operations	App Service/Container Apps, Functions, Storage, SQL, Service Bus, Key Vault, monitoring
Data / database	High	Transactional catalogue storage and reporting support	SQL schema, indexes, constraints, migrations, stored procedures where justified
API / integration	High	UI, supplier and supporting-system connectivity	REST, OpenAPI, auth, pagination, filters, versioning, resilience
QA / test automation	High	Quality expectations	Unit, integration, API, contract, E2E, import fixtures, mocks
DevOps / CI-CD	High	Migration and operational ownership	GitHub Actions, Docker, artifacts, environments, approvals, rollback

Capability	Criticality	What it means in this project	Evidence / deliverables
Business analysis / product	High	Requirements and customer/business alignment	User stories, acceptance criteria, glossary, process maps, prioritisation
HVAC / B2B e-commerce domain	High	Realistic product and commercial behaviour	HVAC attributes, supplier/customer pricing, availability, business rules
Security / identity	Medium	Technology separation and controlled access	RBAC, Entra ID, JWT/cookies, Key Vault, audit, secure headers, validation
Production support	High	Steady-state responsiveness	Monitoring, alerting, dashboards, runbooks, import reprocessing, incident flow

Capability evidence principle
For each capability, the final project should produce something observable: code, configuration, tests, dashboards, a runbook, an architecture decision record, or a working user journey. “We know Azure” is not evidence; a deployed app, monitored service, secure configuration, and documented recovery procedure are.

3. Project scope and business capabilities

3.1 In scope

- Product creation, editing, activation/deactivation, categorisation and search.
- Supplier registration, supplier-specific product identifiers and mapping rules.
- Product attributes, prices by currency, inventory/availability and effective dates.
- Catalogue import from CSV and simulated EDI; supplier REST API synchronization.
- Import history, validation errors, reject/retry/reprocess workflow.
- Role-based admin access and audit history.
- API documentation through OpenAPI/Swagger.
- Automated unit/integration/E2E tests, CI/CD, Docker, logging and monitoring.

3.2 Out of scope for the first release

- Real payment processing
- Real customer checkout
- Real production EDI trading-partner certification
- Enterprise MDM/PIM replacement
- High-volume multi-region active-active deployment
- Complex pricing contracts that depend on an external ERP

3.3 Personas

Persona	Needs	Main capabilities
Catalogue Administrator	Manage products and categories	CRUD, bulk import, approval, audit
Supplier Manager	Manage supplier mappings and integration health	Supplier registry, import history, mapping
Business User / Buyer	Find and inspect catalogue items	Search, filters, product detail, price/availability
Support Engineer	Investigate failures and reprocess imports	Logs, dashboards, failed files, retry/replay
Platform / DevOps Engineer	Deploy and operate the platform	CI/CD, infrastructure, secrets, alerts
Developer	Build and test new capabilities	Local Docker, mocks, fixtures, automated tests

4. Functional requirements and user journeys

FR-01 Product management: Create, read, update, activate and deactivate products. A product has a stable internal ID and a supplier-facing SKU/reference where applicable.

FR-02 Category management: Maintain a hierarchical category tree and associate products with one or more categories.

FR-03 Supplier onboarding: Create a supplier record, declare its integration type, define mapping rules and test connectivity.

FR-04 File import: Upload or receive supplier CSV/EDI files, store the raw artifact, generate an import record, validate and process asynchronously.

FR-05 Import diagnostics: Show file status, counts, processing duration, failed rows, validation messages and reprocessing controls.

FR-06 Supplier API sync: Pull sample product data from external/demo APIs through a supplier adapter without leaking supplier-specific models into the domain.

FR-07 Search: Search by SKU, name, supplier, category and status with pagination and sorting.

FR-08 Security: Role-based permissions for Admin, SupplierManager, Buyer and Support.

FR-09 Audit: Record material changes such as product updates, imports, mapping changes and administrative actions.

FR-10 Operations: Expose health checks, metrics/logging and a support-friendly error model.

4.1 Example end-to-end user journey: supplier catalogue upload

1. Catalogue administrator selects a supplier and uploads products.csv.
2. API validates file metadata and stores the original file in Blob Storage under a quarantine/raw path.
3. An import record is created with status RECEIVED.
4. A queue message is published containing ImportId, SupplierId, BlobPath and CorrelationId.
5. A .NET worker retrieves the message and parses the file in a streaming-friendly manner.
6. Each row is mapped to the canonical ProductImportRow model.
7. Validation checks structure, required fields, SKU uniqueness, category validity, price/currency and supplier mapping rules.
8. Valid rows are upserted transactionally in batches; invalid rows are persisted to ImportItemError.
9. The import status moves to COMPLETED, COMPLETED_WITH_ERRORS or FAILED.
10. The Vue UI refreshes the import dashboard and exposes detailed errors and reprocess controls.

5. Architecture principles and decisions

ID	Principle	Decision	Practical effect
P1	Modular monolith first	Keep the core catalogue in one deployable .NET application with clear modules; extract services only where scaling or	

ID	Principle	Decision	Practical effect
		lifecycle demands it.	
P2	Canonical model	External EDI/CSV/API shapes never become the internal domain model.	
P3	Async for bulk work	Use queues for imports, enrichment and long-running supplier jobs.	
P4	Idempotency everywhere	Retries must not duplicate products, imports or events.	
P5	Secure configuration	No passwords or connection strings committed to source control.	
P6	Observable by design	Every import has ImportId and CorrelationId; logs include supplier and operation context.	
P7	Automate quality gates	A merge should run build, tests and static/security checks before deployment.	
P8	Portable dev setup	Local Docker Compose should provide SQL Server, the API, worker and web app where feasible.	
P9	Replaceable integrations	Each supplier adapter implements an interface so a demo API can be replaced by a real supplier later.	

6. System context architecture

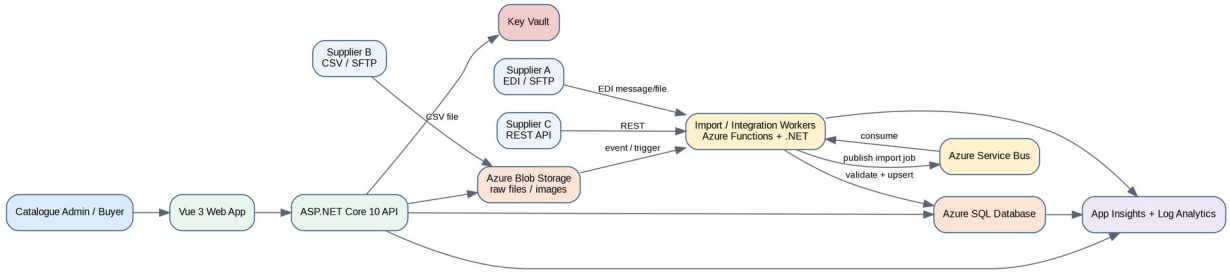


Figure 1 - System context and primary Azure dependencies

6.1 Context explanation

The Vue web app is the primary interactive client. It calls the ASP.NET Core API. Supplier integrations enter through either an integration endpoint or file storage, depending on the transport. The processing layer converts external data into canonical internal records, writes business data to Azure SQL, and keeps raw files in Blob Storage. Service Bus provides asynchronous decoupling, while Key Vault and Application Insights support security and operations.

6.2 Why not connect suppliers directly to the SQL database?

Suppliers should not know the internal database schema. Direct database coupling makes schema changes expensive, bypasses application validation and authorization, and makes integration retry/error handling harder. Suppliers should integrate through a controlled adapter boundary.

7. Logical application architecture

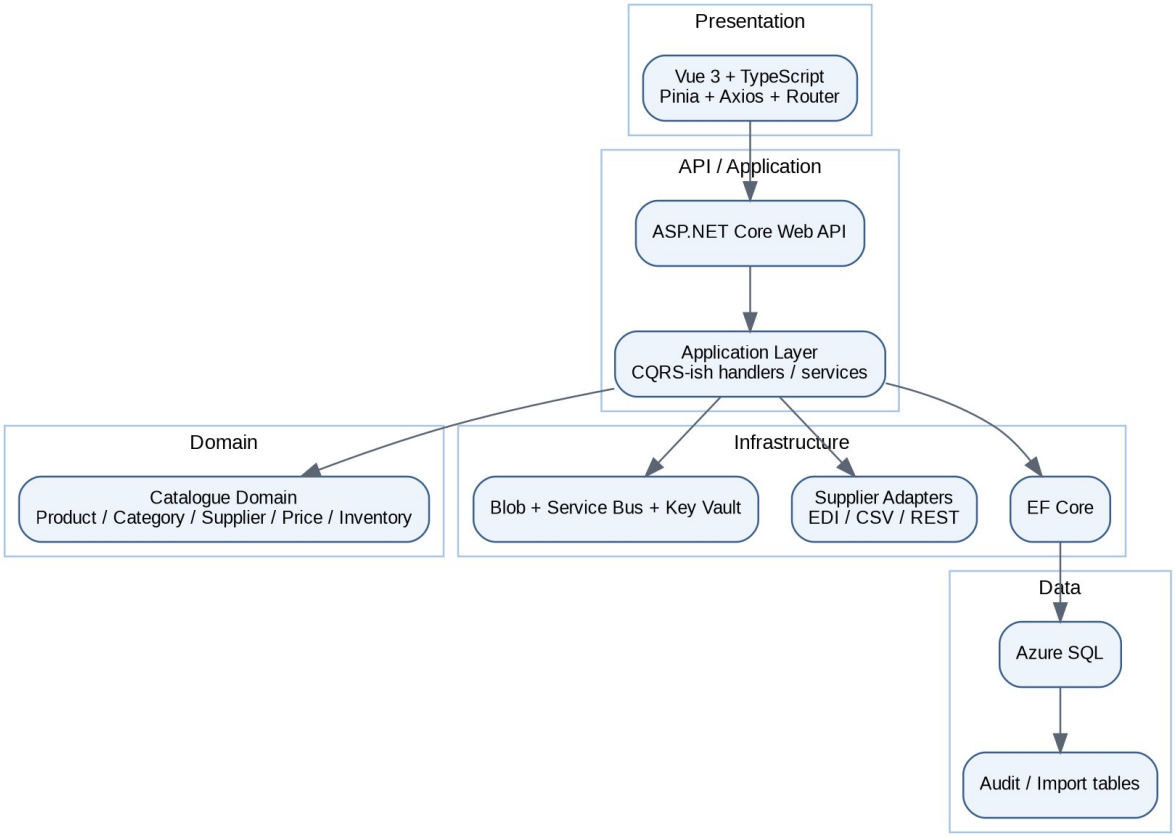


Figure 2 - Logical application architecture

7.1 Recommended solution structure

```
src/
  Catalogue.Web/              # ASP.NET Core host / API
  Catalogue.Application/      # use cases, commands, queries, DTOs
  Catalogue.Domain/           # entities, value objects, business rules
  Catalogue.Infrastructure/   # EF Core, Azure SDKs, supplier adapters
  Catalogue.Workers/          # import / async processing
  Catalogue.Contracts/        # shared API/event contracts
web/
  catalogue-ui/               # Vue 3 + TypeScript
  tests/                      # Playwright/Cypress style E2E

tests/
  Catalogue.UnitTests/
  Catalogue.IntegrationTests/
  Catalogue.ApiTests/
  Catalogue.WorkerTests/
```

7.2 Dependency direction

- Domain depends on nothing external.
- Application depends on Domain and abstractions/interfaces.
- Infrastructure implements Application interfaces and talks to EF Core/Azure/external APIs.

- Web depends on Application + Infrastructure composition root.
- Workers depend on Application + Infrastructure.
- The Vue UI depends on HTTP contracts, not SQL or business logic.

8. Azure deployment architecture

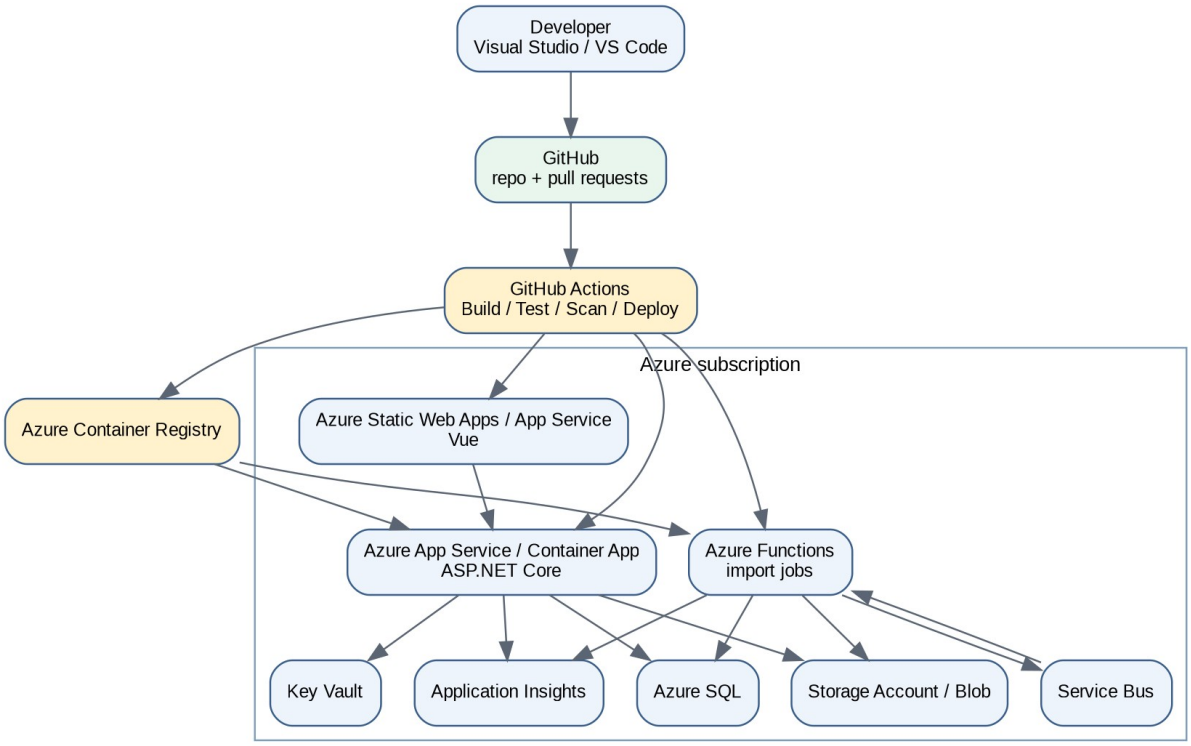


Figure 3 - Deployment and CI/CD architecture

8.1 Azure service mapping

Service	Role	Use in this project
Azure App Service or Container Apps	ASP.NET Core API	Host REST API without managing VMs; container-ready deployment.
Azure Functions	Import triggers / small event-driven jobs	Run event-driven or scheduled processing with less infrastructure management.
Azure Service Bus	Import queue	Decouple ingress from processing, absorb bursts, support retry/dead-letter patterns.
Azure Storage / Blob	Raw EDI/CSV, product images, exports	Durable object storage for unstructured files.
Azure SQL Database	Transactional catalogue data	Managed relational database for products, suppliers, imports and audit.
Azure Key Vault	Secrets, keys, certificates	Central secret/key store; access via managed identity where possible.
Application Insights + Log Analytics	Observability	Application telemetry, logs, dashboards and alerts.
Microsoft Entra ID	Identity	User authentication and enterprise identity integration.
Azure Container Registry (optional)	Container image registry	Store versioned images for API/worker deployments.
Azure API Management (optional)	API gateway / partner-facing APIs	Add when external API governance, policies or partner onboarding justify it.

Lean Azure path
For a portfolio/learning build, start with App Service + Azure SQL + Blob Storage + Functions + Service Bus + Key Vault + Application Insights. Add API Management, private endpoints, multi-region, and a larger container platform only when the project requirements demonstrate the need.

9. Catalogue domain model and database

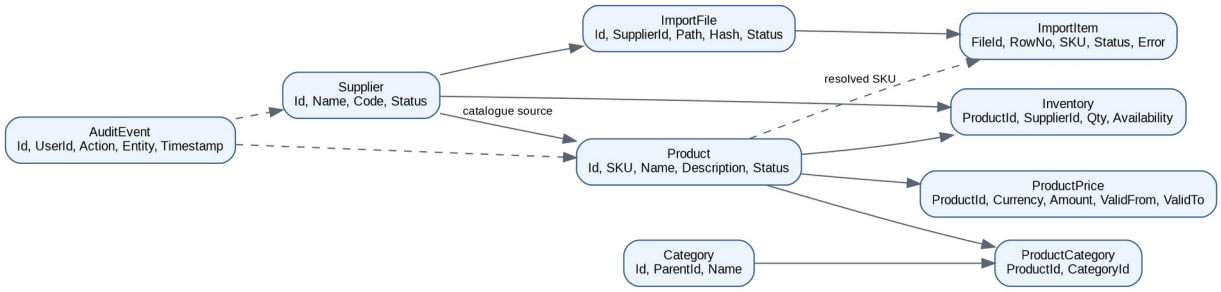


Figure 4 - Simplified relational model

9.1 Core tables

Table	Key columns	Purpose
Product	ProductId, SKU, Name, Description, Status, SupplierId (optional source)	Core product record
Supplier	SupplierId, Code, Name, IntegrationType, Status	External source/company registry
Category	CategoryId, ParentCategoryId, Name, Code, Status	Hierarchical product taxonomy
ProductCategory	ProductId, CategoryId	Many-to-many product taxonomy
ProductPrice	ProductPriceId, ProductId, Currency, Amount, ValidFrom, ValidTo	Effective-dated pricing
Inventory	InventoryId, ProductId, SupplierId, Quantity, Availability, UpdatedAt	Supplier availability
ImportFile	ImportId, SupplierId, BlobPath, FileHash, Format, Status, ReceivedAt, CompletedAt	Import lifecycle
ImportItemError	ImportItemErrorId, ImportId, RowNumber, Field, Message, RawValue	Row-level diagnostics
AuditEvent	AuditEventId, ActorId, Action, EntityType, EntityId, Timestamp, Details	Traceability

9.2 Important database constraints

- Unique constraint on Supplier.Code.
- Unique constraint on Product SKU within the intended business scope (global or supplier-specific, explicitly decided).
- Foreign keys for Supplier, Category and product relationships.
- Check constraints for non-negative quantity and valid status values where appropriate.
- Indexes on SKU, Product.Name (depending on search strategy), SupplierId, CategoryId, ImportFile.Status and ImportFile.ReceivedAt.
- Use UTC timestamps in the database and convert only at the UI boundary.

10. Supplier integration and EDI architecture

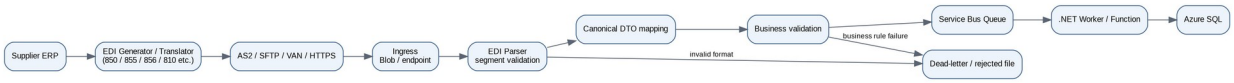


Figure 5 - EDI/file processing path

10.1 What “EDI” means in this project

EDI (Electronic Data Interchange) is a standardized machine-to-machine exchange of business documents. In the learning project, EDI should be treated as an external message format plus an integration boundary. The project can simulate common transaction categories such as purchase orders or catalogue-related messages, but it should not claim to be a certified trading partner implementation.

10.2 EDI simulation strategy

```
Samples/
  supplier-a/
    catalogue-001.edi
    catalogue-002.edi
  supplier-b/
    products.csv

Adapters/
  IRealSupplierAdapter.cs
  EdiSupplierAdapter.cs
  CsvSupplierAdapter.cs
  RestSupplierAdapter.cs
```

10.3 Canonical mapping

```
Supplier EDI / CSV / REST
        |
        v
Supplier DTO (external format)
        |
        v
CanonicalProductImport
        |
        +--> validation
        |
        +--> Product / Price / Inventory entities
```

Enterprise note
Real EDI implementations often involve an agreed standard, trading-partner identifiers, acknowledgements, envelopes, transport/security setup and operational certification. For a learning project, generate deterministic sample EDI and keep the parser isolated so a real EDI library or integration platform can replace it later.

11. File ingestion and transformation pipeline

11.1 State machine

Status	Meaning	Next states
RECEIVED	Raw file stored; metadata captured	PROCESSING, REJECTED
PROCESSING	Worker has locked the import	COMPLETED, COMPLETED_WITH_ERRORS, FAILED
COMPLETED	All valid rows processed	-
COMPLETED_WITH_ERRORS	Some rows rejected but import completed	REPROCESSABLE
FAILED	System-level failure	RETRYING, REPROCESSABLE
REJECTED	File failed structural/security validation	REPROCESSABLE after correction

11.2 Validation layers

11. File-level: extension, size, encoding, checksum, expected columns/EDI envelope, malware scanning in a production scenario.
12. Record-level: required fields, data types, ranges, lengths, enumeration values.
13. Relationship-level: valid supplier, category and identifiers.
14. Business-level: duplicate SKU, invalid price windows, conflicting effective dates, unsupported currency or inactive supplier.
15. Persistence-level: database constraints, concurrency checks and transaction outcomes.

11.3 Idempotency

```
file_hash = SHA256(raw_file_bytes)

if ImportFile exists where SupplierId = supplier_id
and FileHash = file_hash:
   return existing ImportId
else:
   create new import
```

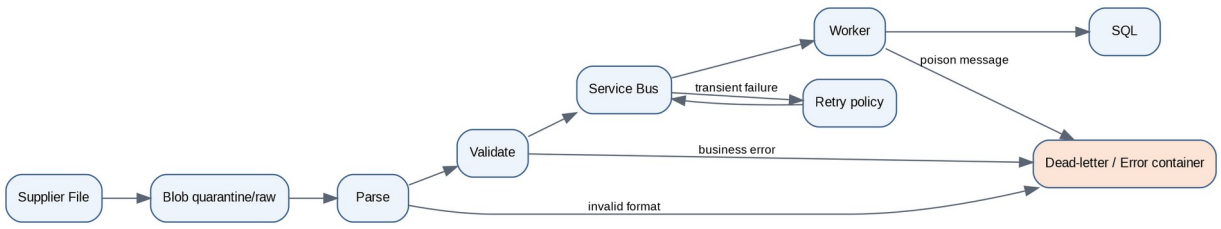


Figure 6 - Failure and recovery flow

12. API architecture and contracts

12.1 REST resources

Endpoint	Purpose	Typical response
GET /api/products	Search/list products	200 + pagination metadata
GET /api/products/{id}	Read product	200 / 404
POST /api/products	Create product	201
PUT /api/products/{id}	Replace/update product	200 / 404 / 409
POST /api/products/{id}/activate	Activate product	200

Endpoint	Purpose	Typical response
POST /api/products/{id}/deactivate	Deactivate product	200
GET /api/categories	List categories	200
POST /api/suppliers	Create supplier	201
POST /api/imports	Create/import a file	202 for async processing
GET /api/imports/{id}	Import status	200
POST /api/imports/{id}/reprocess	Retry/reprocess	202
GET /api/imports/{id}/errors	Import errors	200 + pagination

12.2 Example DTO

```
public sealed record ProductDto(
    long Id,
    string Sku,
    string Name,
    string? Description,
    string Status,
    decimal? Price,
    string? Currency,
    int? AvailableQuantity,
    IReadOnlyList<string> Categories);
```

12.3 HTTP semantics

- 400 = invalid client input
- 401 = unauthenticated
- 403 = authenticated but not allowed
- 404 = resource not found
- 409 = duplicate/conflict
- 422 = semantically invalid domain input (optional convention)
- 500 = unexpected server failure
- 202 = accepted for asynchronous work

13. Vue frontend architecture

13.1 Frontend modules

- Vue 3 + TypeScript
- Vue Router for route protection
- Pinia for shared state only where useful
- Axios or fetch wrapper for API access
- Component-level validation and server-error rendering
- Playwright/Cypress-style E2E tests
- Typed API client generated from OpenAPI as an optional enhancement

13.2 Suggested pages

Route	Purpose	Key components
/login	Authentication	Login form
/catalogue	Browse/search products	Search bar, filters, result grid/table
/products/new	Create product	Product form
/products/:id	View/edit product	Detail, price, inventory, audit
/suppliers	Supplier registry	Supplier list, integration type, health

Route	Purpose	Key components
/imports	Import dashboard	Status, counts, errors, reprocess
/imports/:id	Import detail	File metadata, row errors, timeline
/admin/categories	Category management	Tree/list editor
/support/health	Operational health	Dependencies, recent failures, telemetry links

13.3 Example API call

```
const response = await axios.get('/api/products', {
  params: { search: 'heat pump', page: 1, pageSize: 20 }
});
products.value = response.data.items;
```

14. Security and identity

14.1 Authentication

For a portfolio build, use a replaceable authentication abstraction. In Azure, the production-shaped option is Microsoft Entra ID. Locally, a development identity provider or test token issuer can be used. Avoid implementing a custom password system just to demonstrate authentication unless the learning goal specifically requires it.

14.2 Authorization model

Role	Example permissions
Admin	Full catalogue, supplier, category, import and user-management access
SupplierManager	Supplier and import administration; limited catalogue administration
Buyer	Read catalogue and search; no integration configuration
Support	Read catalogue and imports; reprocess imports; operational diagnostics
Viewer	Read-only catalogue

14.3 Security controls

- Use HTTPS everywhere outside local development.
- Use managed identities from Azure workloads to access Key Vault/Storage where supported.
- Store supplier credentials and API keys in Key Vault, not appsettings committed to Git.
- Validate and constrain uploaded files. Never trust filename or content type alone.
- Apply least privilege to service principals and database users.
- Add correlation IDs and audit events without logging secrets, tokens or full sensitive payloads.
- Use secure headers and CORS allow-lists for the Vue app.
- Use parameterized SQL / EF Core; avoid string concatenation for queries.

15. Testing and mock strategy

15.1 Test pyramid

```
          E2E / UI        small number
        /----------\
   API / integration      moderate
```

```
    /------------------\
  Unit / domain / mapping    large
```

15.2 What to test

Layer	Test	Purpose
Unit	Business rules, validators, mappers	Fast, deterministic
Integration	EF Core + SQL, Blob, Service Bus adapters	Realistic infrastructure
API	HTTP contract, auth, validation, pagination	Black-box service behaviour
Contract	Supplier adapter input/output schemas	Detect integration drift
Worker	Import state transitions, retries, idempotency	Async reliability
E2E	Vue -> API -> database happy paths	Critical user journeys
Performance	Search, imports, concurrent requests	Baseline and regressions

15.3 Mock supplier interface

```
public interface ISupplierProductClient
{
   Task<IReadOnlyList<SupplierProductDto>> GetProductsAsync(
      CancellationToken cancellationToken);
}

public sealed class FakeSupplierProductClient : ISupplierProductClient
{
   public Task<IReadOnlyList<SupplierProductDto>> GetProductsAsync(
      CancellationToken cancellationToken)
      => Task.FromResult<IReadOnlyList<SupplierProductDto>>([
         new("SUP-1001", "Demo Heat Pump", 1299.99m, "GBP", 25)
      ]);
}
```

15.4 Import fixture testing

- Keep a versioned /tests/fixtures directory with valid, malformed, duplicate and boundary-condition files.
- Assert final ImportFile status and exact rejection reasons.
- Verify that running the same file twice produces one effective import, not duplicate business rows.

16. CI/CD, Docker and environment promotion



Figure 7 - CI/CD pipeline

16.1 Multi-stage Dockerfile

```
# Build stage
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet restore
RUN dotnet publish src/Catalogue.Web/Catalogue.Web.csproj -c Release -o /app/publish /p:UseAppHost=false

# Runtime stage
```

```
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "Catalogue.Web.dll"]
```

16.2 Example GitHub Actions pipeline

```
name: build-test

on:
  pull_request:
  push:
    branches: [main]

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.0.x'
      - run: dotnet restore
      - run: dotnet build --no-restore -c Release
      - run: dotnet test --no-build -c Release
      - name: Build container
        run: docker build -t catalogue-api:${{ github.sha }} .
```

16.3 Environment strategy

Environment	Purpose	Data
Local	Developer workstation	Seed data + local containers/mocks
Dev	Shared integration environment	Synthetic/sample data
Test/UAT	Regression and acceptance testing	Representative synthetic data
Prod	Real workloads	Controlled access and production data

17. Observability, support and operations

17.1 Logging

```
logger.LogInformation(
    "Import {ImportId} for supplier {SupplierId} changed to {Status}",
    importId, supplierId, status);
```

17.2 Correlation

Every HTTP request should have a correlation ID. Import jobs should preserve it when a message is published and consumed. The same ImportId should appear in application logs, the import dashboard, and audit events. This lets support trace one business operation across API, Blob, Service Bus and worker processing.

17.3 Support dashboard

Panel	Metric / signal	Why
API health	request rate, 4xx/5xx, latency	User-facing service health
Imports	received, processing, completed, failed	Business processing health
Queue	active count, age, dead letters	Backlog and poison message detection
Database	CPU, connections, query duration	Data-layer bottlenecks
Blob	ingress volume, failed writes	File pipeline health
Dependencies	supplier/API latency and failure rate	External dependency health

18. Reliability, resilience and failure handling

18.1 Failure categories

Failure	Strategy	Design note
Transient network error	Retry with exponential backoff + jitter	Do not retry forever
Supplier API rate limit	Respect Retry-After where provided; backoff	Protect external system
Malformed file	Reject/quarantine	Requires corrected input
One bad row	Continue if business allows; persist row error	Do not lose the rest of the file
Poison queue message	Dead-letter after bounded retries	Manual/automated remediation
Database conflict	Detect 409/constraint conflict; log business key	Requires deterministic resolution
Duplicate file	Hash/idempotency check	Do not process twice
Service outage	Health checks + alert + replay queue/file	Recover after dependency returns

18.2 Retry boundaries

Retry only at boundaries where the operation is safe to repeat. A file download can often be retried; a database write should either be inside a transaction or be made idempotent. A POST to an unknown third-party API should not automatically be retried unless the operation has an idempotency key or equivalent guarantee.

19. Performance and scalability

- Use pagination for product search; never return an unbounded catalogue list.
- Use async EF Core and database indexes matched to actual query patterns.
- Process large imports in bounded batches rather than loading an entire file into memory.
- Use Service Bus to absorb bursty import traffic.
- Cache stable reference data such as categories or country codes when appropriate.
- Avoid N+1 database queries from API endpoints.
- Add client-side debouncing for search boxes in the Vue UI.

19.1 Initial performance targets

Area	Initial target	Notes
Catalogue search API	p95 < 500 ms in a small test dataset	Measure locally and in Dev; tune after evidence
Product detail	p95 < 300 ms	Cache/reference lookups if necessary
Import throughput	> 1,000 rows/min in demo environment	Depends heavily on file size and DB tier
UI initial load	< 3 s on typical broadband/laptop	Bundle size and API waterfalls matter

20. Data governance, audit and compliance considerations

- Separate raw supplier artifacts from canonical business data so the original message can be reprocessed/audited.
- Store data lineage: SupplierId, SourceFile/ImportId, MappingVersion and timestamps.
- Define retention for raw files and audit events rather than keeping everything forever by default.
- Never log credentials, access tokens or unnecessary personal data.
- Document authoritative source of truth for each field, especially price and availability.
- Use versioned mapping rules when suppliers change formats.

20.1 Data lineage example

```
Supplier A
  -> catalogue-2026-09-16.edi
  -> ImportId = 10027
  -> MappingVersion = SUPA-v3
  -> CanonicalProduct SKU = ABC123
  -> ProductId = 8841
  -> ProductPrice + Inventory updated
```

21. Free / public third-party APIs and how to use them

The following options were checked against current public documentation on 16 September 2026. They are useful for prototyping and test data, but “free” does not mean a production SLA or unlimited commercial use. Cache responses, respect rate limits/terms, and keep these dependencies behind adapters so they can be replaced.

API	Use	Docs	Important current note	Project use
DummyJSON	Product catalogue, categories, carts, users, auth	https://dummyjson.com/products	Excellent demo supplier/catalogue API. Supports product search, categories, pagination. POST/PUT/DELETE on products simulate changes rather than persist them.	Use as Supplier-Demo-01 adapter.
Open Food Facts	Real open product catalog data	https://world.openfoodfacts.org/data	Real product data; current API docs recommend v3. Useful if you want richer product attributes.	Use as optional enrichment source; check rate limits and data/licensing requirements.
Open Library	Books/product-like catalogue data and cover images	https://openlibrary.org/developers/api	Public APIs for book/author discovery; identified requests have higher request limits than anonymous use. Not intended as a high-traffic commercial backend.	Good for learning search/enrichment patterns, not as primary production datastore.
REST Countries	Reference data: countries, currencies, languages	https://restcountries.com/docs	Read-only reference APIs; current service uses account/API keys and monthly quota.	Use for country/currency selection and address/reference demos.
ExchangeRate-API Open Access	Daily exchange rates	https://www.exchangerate-api.com/docs/free	Open access endpoint requires attribution, is rate-limited, and updates daily.	Use to demo currency enrichment; cache responses.
JSONPlaceholder	Generic fake REST resources	https://jsonplaceholder.typicode.com/	Free fake REST API for testing/prototyping; supports common HTTP methods and related resources.	Useful for mock external integration patterns, not real catalogue data.

21.1 Recommended integration set for this project

16. DummyJSON as the main demo supplier API because its product endpoints directly support catalogue-like scenarios.
17. Your own local EDI file generator/parser for the enterprise integration capability.
18. Your own CSV fixtures for deterministic data-import tests.
19. Open Food Facts as an optional enrichment adapter if you want real-world product data.
20. Rest Countries and ExchangeRate-API as reference-data/currency services if needed by the UI.

> **Do not make the whole project depend on free APIs**
> A robust architecture uses third-party APIs as replaceable suppliers. Your database and canonical model remain authoritative for the application. Store source identifiers and mapping versions so external data can be re-synchronized safely.

21.2 Current source links

DummyJSON products docs: https://dummyjson.com/docs/products

DummyJSON auth docs: https://dummyjson.com/docs/auth

Open Food Facts API docs: https://openfoodfacts.github.io/documentation/docs/Product-Opener/api/

Open Library API docs: https://openlibrary.org/developers/api

REST Countries docs: https://restcountries.com/docs

ExchangeRate-API free/open access docs: https://www.exchangerate-api.com/docs/free

JSONPlaceholder: https://jsonplaceholder.typicode.com/

22. Sample code and project structure

22.1 Domain entity

```
public sealed class Product
{
    public long Id { get; private set; }
    public string Sku { get; private set; } = null!;
    public string Name { get; private set; } = null!;
    public ProductStatus Status { get; private set; }

    public void Activate() => Status = ProductStatus.Active;
    public void Deactivate() => Status = ProductStatus.Inactive;
}
```

22.2 Import service

```
public async Task<long> StartImportAsync(
    long supplierId,
    string blobPath,
    CancellationToken ct)
{
    var import = new ImportFile(supplierId, blobPath);
    db.ImportFiles.Add(import);
    await db.SaveChangesAsync(ct);
```

```
    await bus.PublishAsync(new ImportRequested(
        import.Id, supplierId, blobPath), ct);

    return import.Id;
}
```

22.3 Supplier adapter

```
public interface ISupplierAdapter
{
    string SupplierCode { get; }
    Task<IReadOnlyList<CanonicalProductImport>> ReadAsync(
        Stream source, CancellationToken ct);
}
```

22.4 Why adapters matter

Without an adapter boundary, the domain layer starts learning every supplier's vocabulary. With adapters, SupplierAProduct, SupplierBRow and EdiSegment are translated into CanonicalProductImport, and the rest of the system only knows the canonical model.

23. End-to-end scenarios

Scenario A - Admin creates product

Vue form -> POST /api/products -> Application validation -> EF Core transaction -> Azure SQL -> audit event -> UI refresh

Scenario B - Supplier CSV

Supplier sends CSV -> Blob raw/quarantine -> Import record -> Service Bus -> Worker parses -> canonical mapping -> validation -> SQL batch upsert -> status/dashboard

Scenario C - Supplier EDI

EDI message/file -> EDI parser -> canonical mapping -> validation -> queue -> worker -> SQL -> acknowledgement/logging

Scenario D - Supplier REST sync

Scheduled Function -> GET supplier API -> supplier DTO -> mapping/validation -> queue if heavy -> SQL upsert -> metrics

Scenario E - Failed import

Parse/validation error -> row/file error persisted -> status failed or completed_with_errors -> support dashboard -> correct source -> reprocess -> idempotency check -> final status

Scenario F - Production incident

Alert -> correlation ID -> Application Insights -> logs/trace -> dependency check -> mitigation -> replay/reprocess -> post-incident record

24. Capability-to-implementation matrix

Capability	Architecture mechanism	Implementation examples	Evidence
Catalogue domain	Domain model + workflows	Product, Category, Price, Inventory	Domain unit tests, UAT scenarios
Supplier integrations	Adapter pattern	EDI/CSV/REST suppliers	Contract tests, connectivity tests
File processing / data transformation	Import pipeline	Blob, parser, mapper, validation	Fixture tests, idempotency tests
Application engineering	Modular monolith	ASP.NET Core + Vue	Code review, tests, API docs
Azure / cloud	PaaS/event-driven	App Service, Functions, SQL, Storage, Service Bus, Key Vault	Deployed environment + IaC/config evidence
Data / database	Relational model	Azure SQL/EF Core	Migration scripts, indexes, query tests
API / integration	REST + events	OpenAPI, Service Bus	API tests, schema validation
QA / test automation	Test pyramid	xUnit, integration tests, Playwright/Cypress-style E2E	Pipeline test report
DevOps / CI-CD	Automated promotion	GitHub Actions + Docker	Successful pipeline + rollback plan
Business analysis / product	Requirements traceability	User stories, acceptance criteria, glossary	Story-to-test traceability
HVAC / B2B e-commerce domain	Domain rules	HVAC attributes, supplier/customer pricing	Domain workshop notes + scenarios
Security / identity	RBAC + secret management	Entra ID, Key Vault, managed identity	Security test cases + least-privilege review
Production support	Observability + runbooks	App Insights, alerts, dashboards, runbooks	Support drill / incident scenario

25. Implementation roadmap

Phase	Outcome	Deliverables
Phase 0	Foundation	Repo, solution, Docker Compose, CI build, coding standards
Phase 1	Catalogue core	Products, categories, suppliers, SQL schema, EF Core, CRUD API, Vue UI
Phase 2	Search + UX	Filtering, pagination, sorting, validation, error handling
Phase 3	CSV ingestion	Blob/local file, ImportFile state machine, parser, mapping, batch upsert
Phase 4	Async processing	Service Bus, worker/function, retry, dead-letter, reprocess
Phase 5	EDI simulation	Sample EDI generator, parser, adapter, mapping, acknowledgement simulation
Phase 6	Supplier REST	DummyJSON adapter, sync job, external-to-canonical mapping
Phase 7	Security	Entra ID shape, roles, Key Vault, secret handling, audit
Phase 8	Testing	Unit/integration/API/E2E tests, fixtures, mock adapters
Phase 9	Observability	Structured logging, Application Insights, dashboard, alerts, runbooks
Phase 10	Production shape	Azure deployment, Docker image, deployment slots/blue-green option, backups/recovery drill

26. Backlog / user stories

ID	Story
CAT-001	As an admin, I can create a product so it appears in the catalogue.

ID	Story
CAT-002	As an admin, I can classify a product into one or more categories.
CAT-003	As a buyer, I can search products by SKU/name and paginate the results.
SUP-001	As a supplier manager, I can register a supplier and choose an integration type.
IMP-001	As a supplier manager, I can upload a CSV and see asynchronous processing status.
IMP-002	As support, I can view row-level import errors and reprocess a corrected file.
EDI-001	As the platform, I can parse a sample EDI document into the canonical import model.
INT-001	As the platform, I can read a demo supplier REST API through an adapter.
SEC-001	As an admin, I can perform admin-only actions and other roles are denied.
OPS-001	As support, I can correlate an import failure from UI to logs using ImportId/CorrelationId.
DEV-001	As a developer, I can start the full stack locally using Docker Compose.
CI-001	As the team, a pull request automatically builds and runs tests.

27. Architecture risks and trade-offs

Risk	Impact source	Priority	Mitigation
Third-party API instability	Demo APIs may change, throttle or disappear	High	Use adapters, timeouts, caching and local fixtures
EDI complexity	Real EDI standards and trading-partner rules can be much larger	High	Keep sample parser isolated; use an established library/platform for real partner onboarding
Overengineering	Too many microservices increase local and operational complexity	Medium	Start modular monolith + workers
Database bottlenecks	Imports can cause lock/contention issues	Medium	Batch writes, indexes, staging tables, measured tuning
Duplicate processing	Retries can repeat an import	High	File hash + ImportId + unique business keys
Secret leakage	API keys in source or logs	High	Key Vault, secret scanning, redaction
Weak observability	Support cannot explain failures	High	Correlation IDs, structured logs, dashboards, import history
Domain ambiguity	Catalogue/HVAC rules may be misunderstood	High	Glossary + examples + acceptance criteria + domain workshops

28. Definition of done

- All functional requirements have acceptance criteria and automated regression tests where practical.
- No secrets are committed to Git.
- API endpoints have OpenAPI documentation and consistent error responses.
- Products, categories, suppliers and imports have audit/history as designed.
- CSV imports support valid, invalid, duplicate and reprocess scenarios.
- EDI sample files can be parsed, validated and transformed into canonical records.

- Supplier REST integration is isolated behind an adapter and can be replaced with a fake client in tests.
- CI builds, tests and container builds successfully on every pull request or main-branch push according to branch policy.
- Azure environment has monitoring, health checks, and a documented recovery/reprocess path.
- Project has architecture diagram, data model, runbook, setup guide and environment variables/secrets documentation.
- Known limitations and free API dependencies are explicitly documented.

29. Recommended learning sequence

Order	Focus	What to practice in this project
1. C# / .NET foundation	DI, configuration, async/await, EF Core, Web API, validation, logging	
2. SQL	schema design, joins, indexes, transactions, stored procedures, migrations	
3. Vue	components, props/events, composables, Pinia, routing, API integration	
4. REST/API integration	HTTP, JSON, auth, pagination, retries, OpenAPI	
5. File processing	CSV, streaming, validation, idempotency, import state machine	
6. EDI	segments, envelopes, transaction types, parser -> canonical model, transport concepts	
7. Azure	App Service, Functions, Blob, SQL, Service Bus, Key Vault, Application Insights	
8. Docker	Dockerfile, multi-stage build, Compose, networks, volumes, environment variables	
9. CI/CD	GitHub Actions, test gates, image build, registry, deployment environments	
10. Production support	logging, metrics, alerts, tracing, dead-letter/reprocessing, runbooks	

30. Reference links

Current platform/API references consulted for this document (checked 16 September 2026):

Microsoft .NET support policy: https://dotnet.microsoft.com/en-us/platform/support/policy

Azure App Service overview: https://learn.microsoft.com/en-us/azure/app-service/overview

Azure Functions overview: https://learn.microsoft.com/en-us/azure/azure-functions/functions-overview

Azure Service Bus overview: https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-messaging-overview

Azure Blob Storage overview: https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-overview

Azure SQL Database overview: https://learn.microsoft.com/en-us/azure/azure-sql/database/sql-database-paas-overview?view=azuresql

Azure Key Vault overview: https://learn.microsoft.com/en-us/azure/key-vault/general/about-keys-secrets-certificates

DummyJSON product docs: https://dummyjson.com/docs/products

DummyJSON auth docs: https://dummyjson.com/docs/auth

Open Food Facts API: https://openfoodfacts.github.io/documentation/docs/Product-Opener/api/

Open Library APIs: https://openlibrary.org/developers/api

REST Countries docs: https://restcountries.com/docs

ExchangeRate-API free/open access: https://www.exchangerate-api.com/docs/free

JSONPlaceholder: https://jsonplaceholder.typicode.com/

Appendix A - Suggested repository layout

```
.
├── src/
│   ├── Catalogue.Web/
│   ├── Catalogue.Application/
│   ├── Catalogue.Domain/
│   ├── Catalogue.Infrastructure/
│   ├── Catalogue.Workers/
│   └── Catalogue.Contracts/
├── web/
│   └── catalogue-ui/
├── tests/
│   ├── Catalogue.UnitTests/
│   ├── Catalogue.IntegrationTests/
│   ├── Catalogue.ApiTests/
│   └── Catalogue.WorkerTests/
├── samples/
│   ├── edi/
│   ├── csv/
│   └── supplier-json/
├── infra/
│   ├── bicep/
│   └── scripts/
├── docker-compose.yml
├── Dockerfile
├── .github/workflows/ci.yml
└── README.md
```

Appendix B - Environment variable examples

```
ConnectionStrings__CatalogueDb=...
AzureStorage__BlobEndpoint=...
ServiceBus__FullyQualifiedNamespace=...
KeyVault__Uri=...
SupplierApis__DummyJsonBaseUrl=https://dummyjson.com
AllowedOrigins__0=https://<frontend-host>
```

Final architectural recommendation
Build the first vertical slice all the way through: Vue product screen -> .NET API -> SQL -> audit/logging -> CI. Then add supplier CSV -> Blob -> queue -> worker -> SQL. Then add simulated EDI and a demo supplier REST adapter. This order makes each capability visible and testable instead of creating a large unfinished platform.